

AVANCE 1: CONTRATOS, PRUEBAS Y UN REPO QUE CORRE

Qué se entrega, cómo se califica, la plantilla del README y la lista pública de preguntas de la revisión técnica. Léanlo completo antes de la Sesión 13 y prepárense con él.

MODALIDAD	CUÁNDO	PESO	META
Equipo de 3 (nota grupal + en vivo)	Sesión 13 revisión en clase	10 % de la nota final	Sepolia + ≥ 80 % + README

EN UNA FRASE

El Avance 1 demuestra que la **capa de contratos** de su proyecto existe, está probada y cualquiera puede reproducirla: contratos **desplegados y verificados en Sepolia**, una suite de pruebas con **cobertura de líneas ≥ 80 %** y un **repositorio documentado** que otra persona puede clonar y correr sin preguntarles nada. El frontend completo NO es parte de esta entrega: es el Avance 2 (Sesión 16).

01 Qué se entrega

TRES COMPONENTES

COMPONENTE	REQUISITO VERIFICABLE	CÓMO LO COMPRUEBA EL DOCENTE
1 · Contratos en Sepolia	Todos los contratos del proyecto desplegados en Sepolia con Hardhat Ignition y verificados : el código fuente se lee en Etherscan.	Abre los enlaces del README. En la pestaña <i>Contract</i> debe aparecer la marca de verificado.
2 · Suite de pruebas	Pruebas automatizadas con <code>npx hardhat test --coverage</code> que cubren ≥ 80 % de las líneas de SUS contratos, incluidos casos de reversión, límites y eventos.	Les pide correr la cobertura en vivo y señalar una línea no cubierta.
3 · Repositorio documentado	README según la plantilla de la sección 4, sin secretos, con <code>package-lock.json</code> , instrucciones que funcionan desde cero y el análisis de Slither documentado.	Clona el repositorio en una carpeta nueva y sigue el README al pie de la letra.

Cómo se entrega

1. Etiqueten el commit que se revisa y súbanlo: `git tag avance-1` y `git push origin avance-1`.
2. Registren en el formulario del curso: enlace al repositorio, nombre de la etiqueta, direcciones de los contratos en Sepolia y los tres integrantes.
3. Plazo: el día de la Sesión 13, antes de las 23:59. Lo que se revisa es el commit etiquetado; cambios posteriores no cuentan.

QUÉ NO ES ESTA ENTREGA

No es el frontend terminado (Avance 2, Sesión 16). No cuenta la cobertura de librerías externas como OpenZeppelin: solo la de **sus** contratos. Y no se aceptan contratos desplegados «a mano» desde Remix sin un módulo de Ignition en el repositorio: el despliegue tiene que ser reproducible.

02 Rúbrica

ESCALA DE 0 A 5,0

Deriva de la rúbrica final del proyecto (plan de estudio, sección 8): los criterios «Pruebas y despliegue» y «Sustentación y documentación», aplicados a lo que existe en este punto del semestre.

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (< 3,0)
Contratos desplegados y verificados	30 %	Todos desplegados con Ignition y verificados; direcciones y enlaces en el README; el código desplegado coincide con el del commit etiquetado.	Desplegados y verificados, pero sin Ignition o con direcciones que hay que buscar.	Algún contrato sin desplegar o sin verificar.
Pruebas y cobertura	35 %	≥ 80 % de líneas; cada error personalizado y cada modificador de acceso tiene una prueba de reversión; se prueban eventos y límites (cero, máximo, tiempo).	≥ 80 % de líneas, pero casi todo son casos felices.	Menos de 60 %, pruebas que fallan o que no prueban nada (sin <code>expect</code>).
Repositorio y README	25 %	Se instala, prueba y despliega siguiendo el README sin ayuda; sin secretos; Slither documentado en <code>docs/SEGURIDAD.md</code> ; lo centralizado declarado.	Funciona con uno o dos tropiezos menores; documentación incompleta.	No arranca siguiendo el README, o hay una clave o URL con API key en el historial.
Revisión técnica en vivo	10 %	Los tres integrantes responden con solvencia sobre cualquier parte.	Responde bien una sola persona del equipo.	Nadie explica su propio código.

DOS REGLAS QUE NO ADMITEN EXCEPCIÓN

1. Una clave privada, una frase semilla o una URL de RPC con API key en cualquier commit del repositorio deja el criterio «Repositorio y README» en 0,0, aunque después se haya borrado: sigue en la historia de Git. Los secretos van solo en el keystore de Hardhat (`npx hardhat keystore set ...`).
2. Ningún dato personal en cadena (Ley 1581 de 2012). Si su proyecto maneja documentos o personas, en la cadena va solo un hash.

03 Lista de chequeo antes de entregar

MÁRQUENLA EN EQUIPO

✓	ÍTEM
☐	<code>git clone</code> en una carpeta nueva → <code>npm install</code> → <code>npx hardhat test</code> pasa completo. Lo hace un integrante distinto al que más programó.
☐	<code>npx hardhat test --coverage</code> muestra ≥ 80 % de líneas en cada contrato propio. Capturaron la tabla para el README.
☐	Cada <code>error</code> personalizado y cada modificador (<code>onlyOwner</code> , <code>whenNotPaused</code> ...) tiene al menos una prueba con <code>revertedWithCustomError</code> .
☐	Hay al menos una prueba de evento (<code>.to.emit(...).withArgs(...)</code>) por cada evento que la interfaz vaya a leer.
☐	Slither corrido; cada hallazgo alto o medio está corregido o justificado en <code>docs/SEGURIDAD.md</code> .
☐	Módulo(s) de Ignition en <code>ignition/modules/</code> ; parámetros de Sepolia en un JSON sin secretos.
☐	Contratos verificados en Sepolia; direcciones y enlaces en el README; bloque de despliegue anotado (sirve para <code>queryFilter</code>).
☐	<code>package-lock.json</code> en el repositorio; <code>.gitignore</code> con <code>node_modules</code> , <code>artifacts</code> , <code>cache</code> , <code>coverage</code> , <code>.env</code> .
☐	<code>git log -p findstr /i "mnemonic alchemy.com/v2 infura.io/v3"</code> (Windows) o <code>git log -p grep -iE "mnemonic alchemy.com/v2 infura.io/v3"</code> no muestra nada, y ninguna cadena <code>0x</code> de 64 caracteres del historial es una clave (los hashes de transacción también miden 64: revísenlas).
☐	El README declara qué partes quedan centralizadas (RPC, hosting, indexador, administrador del contrato).
☐	Etiqueta <code>avance-1</code> creada y empujada; formulario enviado con las direcciones.

Reemplacen lo que está entre `<...>`. No borren secciones: si una no aplica, escriban «No aplica, porque...». El README es lo primero que el docente, el equipo auditor y un futuro empleador leen.

```
# <Nombre del proyecto>

<Una frase: qué problema resuelve y para quién.>

## 1. Por qué blockchain (y por qué no una base de datos)
<3-5 líneas del anteproyecto: contra quién se protege el sistema, qué
propiedad aporta la cadena, qué alternativa centralizada se descartó y por qué.>

## 2. Arquitectura
<Diagrama o lista: contratos, quién llama a qué, qué emite cada evento.>
| Contrato | Responsabilidad | Patrones aplicados |
|---|---|---|

## 3. Despliegue en Sepolia
| Contrato | Dirección | Etherscan (verificado) | Bloque de despliegue |
|---|---|---|---|
| <Nombre> | 0x... | https://sepolia.etherscan.io/address/0x...#code | <n> |

## 4. Requisitos
- Node.js 22 LTS, Git. Versiones fijadas en package.json / package-lock.json.

## 5. Instalar y probar
  npm install
  npx hardhat test
  npx hardhat test --coverage

<Pegar aquí la tabla de cobertura: archivo · % de líneas.>

## 6. Desplegar
  npx hardhat keystore set SEPOLIA_RPC_URL
  npx hardhat keystore set SEPOLIA_PRIVATE_KEY
  npx hardhat keystore set ETHERSCAN_API_KEY
  npx hardhat ignition deploy ignition/modules/<Modulo>.js --network sepolia --verify

## 7. Seguridad
- Resultado de Slither y qué se hizo con cada hallazgo: ver docs/SEGURIDAD.md
- Roles con privilegios (owner, pausador...) y qué pueden hacer.
- Riesgos conocidos y mitigaciones.

## 8. Qué queda centralizado
<RPC, hosting del frontend, gateway IPFS, indexador, cuenta administradora...
y cómo se descentralizaría en una versión real.>

## 9. Equipo y uso de IA
<Integrantes y rol. Declaración de las herramientas de IA usadas y para qué.>

> Material de curso, sin auditar. Solo redes de prueba; nunca fondos reales.
```

05 La revisión técnica: las preguntas, públicas

En la Sesión 13 el docente pasa por cada equipo unos cinco minutos, con el repositorio, la terminal y Etherscan abiertos. Hará **algunas** de estas preguntas, a **cualquiera** de los tres integrantes. No hay preguntas sorpresa: todas están aquí. Quien construyó su parte, las responde sin esfuerzo; quien no, se nota en la primera.

#	TEMA	PREGUNTA
1	Contrato	Abran su contrato verificado en Etherscan. ¿Qué hace esta función, línea por línea? ¿Quién puede llamarla?
2	Contrato	¿Qué patrón de diseño aplicaron (retiro, control de acceso, pausa, máquina de estados...) y qué problema concreto resuelve en su proyecto?
3	Pruebas	Corran la cobertura ahora. ¿Qué línea NO está cubierta? ¿Por qué aceptan ese riesgo o cuándo la cubrirán?
4	Pruebas	Muéstrenme una prueba que espere una reversión. ¿Qué error personalizado verifica y qué pasaría si la condición no existiera?
5	Pruebas	¿Qué caso límite probaron (cero, máximo, fin de plazo, llamada repetida)? ¿Cuál se les ocurre ahora que no probaron?
6	Seguridad	¿Qué dijo Slither? Elijan un hallazgo y expliquen qué hicieron con él.
7	Seguridad	¿Qué cuenta tiene privilegios en su contrato? Si esa clave se filtra, ¿qué puede hacer un atacante?
8	Despliegue	¿Cómo desplegaron? Muéstrenme el módulo de Ignition. Si lo corro otra vez, ¿qué pasa?
9	Despliegue	Muéstrenme una transacción real de su contrato en Sepolia. ¿Qué evento emitió y qué dicen sus argumentos?
10	Repositorio	Si clono el repositorio ahora, ¿qué comando corro primero? ¿Dónde están las claves y cómo sé que no están en Git?
11	Diseño	¿Qué parte de su proyecto NO está descentralizada? ¿Lo dice el README?
12	Diseño	¿Qué dato de su sistema NO debe ir en la cadena, y dónde lo guardan? (Pista: Ley 1581.)

CÓMO PREPARARSE (15 MINUTOS EN EQUIPO)

Cada integrante responde en voz alta tres preguntas al azar sobre la parte que NO programó. Si alguno se queda en blanco, esa es la parte que el equipo tiene que explicarse antes de la sesión. La nota de la revisión es del equipo, pero se construye con las respuestas de cada uno.

06 Errores que más puntos cuestan

ERROR	POR QUÉ CUESTA Y CÓMO EVITARLO
«En mi máquina sí corre»	El README omite un paso (una variable del keystore, una versión de Node). Hagan la prueba del clon limpio con otro integrante.
Cobertura inflada	Pruebas que llaman funciones sin <code>expect</code> : suben el porcentaje y no prueban nada. En la revisión se nota en la pregunta 4.
Verificado, pero otro código	Se desplegó una versión y luego se cambió el contrato. Vuelvan a desplegar desde el commit etiquetado.
Secretos en la historia	Un <code>.env</code> subido y luego borrado sigue en Git. Si pasó: roten la clave (billetera nueva del curso) y reescriban la historia con ayuda del docente.
Todo en el storage	Arreglos que guardan historial «para la interfaz». Emitan eventos y léanlos con <code>queryFilter</code> (Sesión 13).